

ViewModel

За по-добра производителност избягваме ViewData и ViewBag като девелъпъри. Все още използваме обаче ViewBag за PageTitle. Начинът едновременно да предадем данни и за PageTitle + сложен тип данни като клас Employee, е да ги обединим в специфичен модел ViewModel, който ще ни позволи включване на Intellisense.

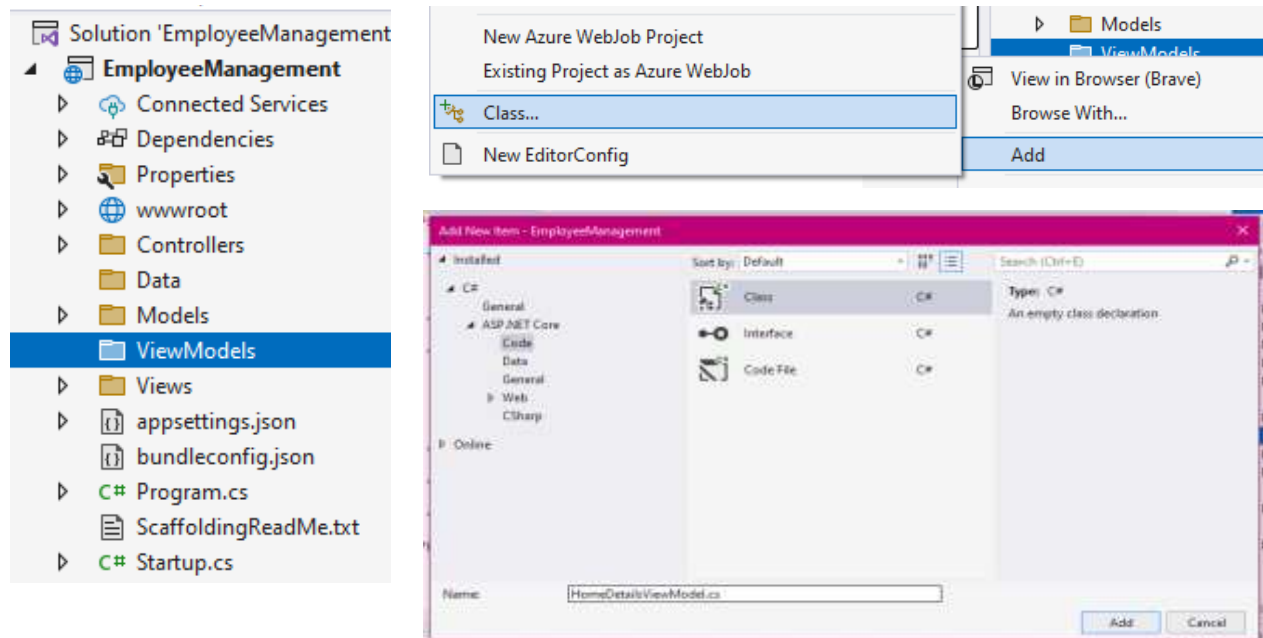
В момента в HomeController използваме все още 2 типа данни за прехвърляне:

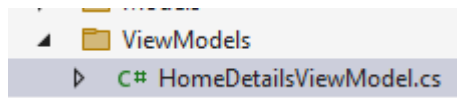


```
0 references
public ViewResult DetailsPageTable()
{
    2 типа Employee model = _employeeRepository.GetEmployee(1);
    ViewData["PageTitle"] = "Employee Details";
    ViewData["Employee"] = model;
    return View(model);
}
```

Няма как на тип Employee да присвоим PageTitle, затова използваме новия тип ViewModel, който ще ги обедини двата в по-сложния тип. Той може да бъде поставен навсякъде в проекта, но за по-добра организация създаваме отделна папка ViewModels. Въпросът е не просто да се обединят двата типа данни в един нов, а да включим Intellisense, за да предотвратяваме грешки по време на писането на кода.

HomeDetailsViewModel.cs = PageTitle + Employee

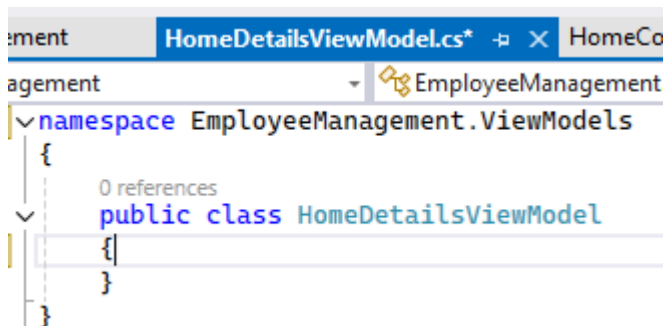




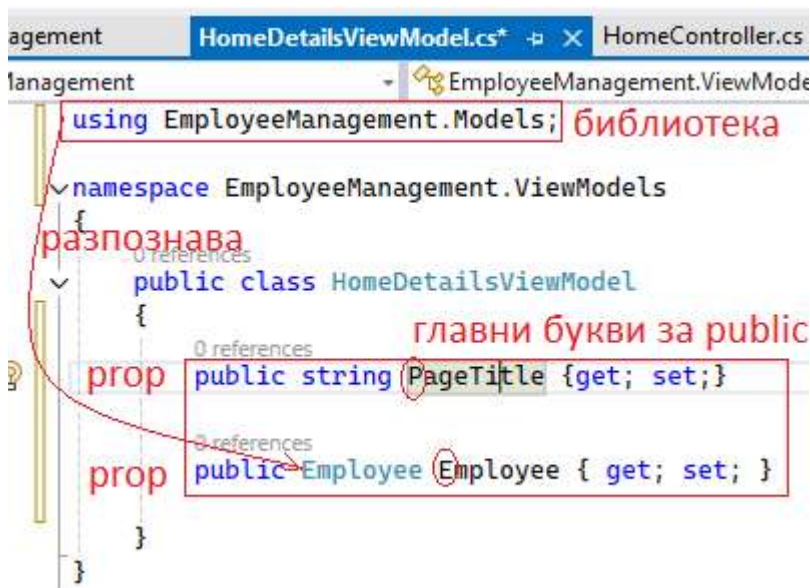
Име **HomeController+Details** action method + **ViewModel** = **HomeDetailsViewModel.cs**
за пренос на данни между контролера и View.

ViewModels = Data Transfer Objects = DTO.

A namespace is a declarative region that provides a scope to the identifiers (the names of types, functions, variables, etc) inside it.



Винаги първо създаваме „библиотеката“, за да може после да я извикаме като декларация с възможност корекции на грешки в контролера или където е необходимо.



Ако пропуснем да напишем къде се намира нашия обект Employee – в папката на проекта Models, той няма да бъде разпознат:


```

{
    public class HomeDetailsViewModel
    {
        public Employee Employee { get; set; }
        public string PageTitle { get; set; }
    }
}

```

В HomeController правим нов метод, който да използва новия модел на по-сложния обект HomeDetailsViewModel – копираме стария DetailsPageTable ->DetailsView

The screenshot shows the Visual Studio IDE with the following components:

- Top Tabs:** HomeController.cs*, HomeDetailsViewModel.cs, DetailsPageTable.cshtml.
- Left Pane:** EmployeeManagement.Controllers.HomeCon, DetailsView().
- Main Editor (HomeController.cs):**

```

0 references
public IActionResult DetailsPageTable()
{
    Employee model = _employeeRepository.GetEmployee(1);
    ViewData["PageTitle"] = "Employee Details";
    ViewData["Employee"] = model;
    return View(model);
}

```

This code block is highlighted with a red box and labeled "copy".
- IntelliSense:**

Below the first code block, the word "Home" is typed, triggering an IntelliSense dropdown. The dropdown lists:

 - HomeController
 - HomeDetailsViewModel (highlighted with a red box and labeled "IntelliSense разпознава")
 - Views_Home_DetailsPage
 - Views_Home_DetailsPageTable
 - Views_Home_Different
 - Views_Home_TestPage
- Main Editor (HomeController.cs):**

```

0 references
public IActionResult DetailsView()
{
    HomeDetailsViewModel model = new HomeDetailsViewModel();
    return View(model);
}

```

This code block is highlighted with a red box and labeled "paste".
- IntelliSense:**

Below the second code block, the word "model" is typed, triggering an IntelliSense dropdown. The dropdown lists:

 - model (highlighted with a blue box and labeled "предположено")
 - homeDetailsViewModel
 - homeDetailsView

Макар, че ако потвърдим с табуляция, този код ще работи

```

public IActionResult DetailsView()
{
    HomeDetailsViewModel model = new HomeDetailsViewModel();
    return View(model);
}

```

по-добре е да използваме стандарта, при който променливата е същата като типа данни, но вместо с главна буква като класа, с малка:

```
0 references
public ActionResult DetailsView()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel();
    return View();
}
```

Тя вече е разпознаваема и може да бъде прихваната за връщане (вместо model):

```
0 references
public ActionResult DetailsView()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel();
    return View(home);
}
0 references
homeDetailsViewModel (local variable) HomeC
```

```
0 references
public ActionResult DetailsView()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel();
    return View(homeDetailsViewModel);
}
```

Тази нова променлива, все още е празна и няма стойности, затова трябва да ѝ ги придадем – кодът много прилича на предходния метод:

```
public ActionResult DetailsPageTable()
{
    Employee model = _employeeRepository.GetEmployee(1);
    ViewData["PageTitle"] = "Employee Details";
    ViewData["Employee"] = model;
    return View(model);
}
```

```
0 references
public ActionResult DetailsView()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
    };
    return View(homeDetailsViewModel);
}
```

изтриваме ;
единствен метод

Нашият сложен обект Employee има още заглавие на страница:


```
public ViewResult DetailsView()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
    }
    return View(homeDetailsViewModel);
}
```

А) дори да сбъркаме, коректорът подсказва:

```
public ViewResult DetailsView()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details View"();
    }
    return View(homeDetailsViewModel);
}
```

дори да сгрешим, се автокоригира позицията

0 references

```
public ViewResult DetailsView()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details View"
    };
    return View(homeDetailsViewModel);
}
```

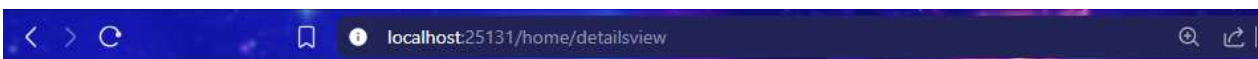
създаване

зареждаме данни за новата променлива

извикване

0 references

```
public ViewResult DetailsView()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details View"
    };
    return View(homeDetailsViewModel);
}
```



An unhandled exception occurred while processing the request.

InvalidOperationException: The view 'DetailsView' was not found. The following locations were searched:
 /Views/Home/DetailsView.cshtml **трябва да го създадем**
 /Views/Shared/DetailsView.cshtml
 /Pages/Shared/DetailsView.cshtml

Model няма свойствата Name, Email, Department. Те принадлежат на Employee, който е едно ниво под него, защото вътре в самия HomeDetailsViewModel = PageTitle + Employee(Name, Email, Department) и използваме подсказките, за да ги присвоим, т.е. не можем да го прескочим като тип вътре в друг тип.

Също вече не е необходимо да прехвърляме PageTitle чрез ViewBag, защото сме го направили част от декларирания в началото Model, към който също е включен Intellisense.

```
r.cs | DetailsView.cshtml | HomeDetailsViewModel.cs | DetailsPageTable.cshtml
management
@model EmployeeManagement.ViewModels.HomeDetailsViewModel
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DetailsPage</title>
</head>
<body>
  <h1>DTO - Data transfer object</h1>
  <hr> HomeDetailsViewModel = PageTitle + Employee
  <h3>@Model.PageTitle : Model - Strongly typed view</h3>
  <div>Id: @Model.Employee.Id</div>
  <div>Name: @Model.Employee.Name</div>
  <div>Family: @Model.Employee.Family</div>
  <div>Phone: @Model.Employee.Phone</div>
  <div>Email: @Model.Employee.Email</div>
  <div>Department: @Model.Employee.Department</div>
</body>
</html>
```

къде
папка модел
Id, Name, Family, Phone, Email, Department are properties of Employee object

```
@model EmployeeManagement.ViewModels.HomeDetailsViewModel
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>DetailsPage</title>
</head>
<body>
  <h1>DTO - Data transfer object</h1>
  <hr>
  <h3>@Model.PageTitle : Model - Strongly typed view</h3>
  <div>Id: @Model.Employee.Id</div>
  <div>Name: @Model.Employee.Name</div>
  <div>Family: @Model.Employee.Family</div>
  <div>Phone: @Model.Employee.Phone</div>
  <div>Email: @Model.Employee.Email</div>
  <div>Department: @Model.Employee.Department</div>
</body>
</html>
```

DTO - Data transfer object

Employee Details View : Model - Strongly typed view

Id: 1

Name: Мария

Family: Георгиева

Phone: 0123456789

Email: Maria@omg.bg

Department: Biology

Така страницата зарежда и PageTitle + Details.

We create a “View Model” when a Model object does not contain all the data a view needs

```
public class HomeDetailsViewModel
{
    public Employee Employee { get; set; }
    public string PageTitle { get; set; }
}
```

```
public ActionResult Details()
{
    HomeDetailsViewModel homeDetailsViewModel = new HomeDetailsViewModel()
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details"
    };

    return View(homeDetailsViewModel);
}
```